

Navya reddy

AIML

AI-Based Virtual Try-On System Using FastAPI and Computer Vision

1. Introduction

The fashion e-commerce industry is rapidly adopting Artificial Intelligence to improve customer experience. One major challenge in online shopping is that customers cannot physically try clothes before purchasing them.

To solve this problem, we developed an **AI Virtual Try-On System** that allows users to upload:

- a person image
- a clothing image

and generate a virtual try-on output showing how the cloth appears on the person.

This project uses:

- Computer Vision
- Image Processing
- FastAPI
- OpenCV
- Python

to simulate a basic AI clothing try-on pipeline.

2. Objective

The main objective of this project is:

- To build an AI-powered virtual try-on prototype
 - To automate garment overlay on human images
 - To explore computer vision techniques in fashion AI
 - To develop an API-based ML application using FastAPI
-

3. Technologies Used

Technology	Purpose
Python	Backend development
FastAPI	API framework
OpenCV	Image processing
NumPy	Numerical operations
Uvicorn	ASGI server
Swagger UI	API testing

4. System Workflow

The workflow of the system is:

1. User uploads person image
 2. User uploads clothing image
 3. System preprocesses images
 4. Cloth image is resized and aligned
 5. Background masking is applied
 6. Garment overlay is generated
 7. Final output image is saved
-

5. Methodology

5.1 Image Upload

FastAPI handles image uploads using REST API endpoints.

5.2 Cloth Preprocessing

The cloth image is:

- resized
- background removed
- masked for better blending

5.3 Virtual Try-On

Using OpenCV:

- shirt position is estimated
- garment is aligned to torso region
- cloth is blended with the person image

5.4 Output Generation

The final try-on image is generated and stored automatically.

6. Features Implemented

FastAPI backend

REST API endpoints

Swagger API documentation

Image upload support

Garment overlay pipeline

Transparent PNG support

Dynamic cloth resizing

Output image generation

These challenges were partially resolved using:

- HSV masking
 - alpha channel masking
 - image blending
 - morphological operations
-

8. Future Improvements

Future versions can include:

- Human pose estimation
- Body segmentation
- Cloth warping
- Diffusion models
- HR-VITON integration
- IDM-VTON integration

9. Conclusion

The AI Virtual Try-On System successfully demonstrates a prototype virtual fitting solution using computer vision and FastAPI.

The project helped in understanding:

- API development
- Image processing
- AI fashion applications
- Computer vision pipelines

Although the current implementation uses OpenCV-based overlay techniques, it provides a strong foundation for integrating advanced AI-based virtual try-on architectures in future development.

Code:

```
from fastapi import FastAPI, UploadFile, File
import shutil
import os
import cv2
import numpy as np
os.makedirs("inputs", exist_ok=True)
os.makedirs("outputs", exist_ok=True)
@app.get("/")
def home():
    return {
        "message": "AI Virtual Try-On Running"
    }
def virtual_tryon(self, person_path, cloth_path):

    person = cv2.imread(person_path)

    cloth = cv2.imread(cloth_path)
```

```

if person is None:
    raise Exception("Person image not found")

if cloth is None:
    raise Exception("Cloth image not found")

person_h, person_w = person.shape[:2]

    gray_person = cv2.cvtColor(
        person,
        cv2.COLOR_BGR2GRAY
    )

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

faces = face_cascade.detectMultiScale(
    gray_person,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(60, 60)
)

if len(faces) == 0:
    raise Exception("Face not detected")

    x, y, w, h = max(
        faces,
        key=lambda f: f[2] * f[3]
    )

    face_center_x = x + w // 2

chin_y = y + h

shoulder_y = int(
    chin_y + (0.45 * h)
)

    left_shoulder_x = int(
        face_center_x - (1.10 * w)
    )

right_shoulder_x = int(
    face_center_x + (1.10 * w)
)

```

```

body_width = right_shoulder_x - left_shoulder_x

        cloth_width = int(body_width * 1.22)

aspect_ratio = cloth.shape[0] / cloth.shape[1]

cloth_height = int(
    cloth_width * aspect_ratio
)

        cloth = cv2.resize(
            cloth,
            (cloth_width, cloth_height)
        )

x_offset = left_shoulder_x - 15

y_offset = shoulder_y - 18

        x_offset = max(0, x_offset)

y_offset = max(0, y_offset)

if x_offset + cloth_width > person_w:
    cloth_width = person_w - x_offset

if y_offset + cloth_height > person_h:
    cloth_height = person_h - y_offset

# Resize again safely
cloth = cv2.resize(
    cloth,
    (cloth_width, cloth_height)
)

        roi = person[
            y_offset:y_offset + cloth_height,
            x_offset:x_offset + cloth_width
        ]

        gray = cv2.cvtColor(
            cloth,
            cv2.COLOR_BGR2GRAY
        )

_, thresh = cv2.threshold(
    gray,
    250,
    255,
    cv2.THRESH_BINARY_INV

```

```

)

    contours, _ = cv2.findContours(
thresh,
cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE
)

    mask = np.zeros_like(gray)

    largest_contour = max(
contours,
key=cv2.contourArea
)

cv2.drawContours(
mask,
[largest_contour],
-1,
255,
thickness=cv2.FILLED
)
kernel = np.ones((5, 5), np.uint8)

mask = cv2.morphologyEx(
mask,
cv2.MORPH_CLOSE,
kernel
)

mask = cv2.GaussianBlur(
mask,
(5, 5),
0
)

    foreground = cv2.bitwise_and(
cloth,
cloth,
mask=mask
)
alpha = mask.astype(float) / 255.0

alpha = cv2.merge([
alpha,
alpha,
alpha
])

foreground = foreground.astype(float)

```

```

        background = roi.astype(float)

        combined = (
            foreground * alpha +
            background * (1 - alpha)
        )
        combined = cv2.GaussianBlur(
            combined,
            (3, 3),
            0
        )

        combined = combined.astype(np.uint8)
        person[
            y_offset:y_offset + cloth_height,
            x_offset:x_offset + cloth_width
        ] = combined

    return person

@app.post("/ai/virtual-tryon")
async def tryon(
    person: UploadFile = File(...),
    cloth: UploadFile = File(...)
):

    # Save person image
    person_path = f"inputs/{person.filename}"

    with open(person_path, "wb") as buffer:
        shutil.copyfileobj(
            person.file,
            buffer
        )
    cloth_path = f"inputs/{cloth.filename}"

    with open(cloth_path, "wb") as buffer:
        shutil.copyfileobj(
            cloth.file,
            buffer
        )

    result = model.virtual_tryon(
        person_path,
        cloth_path
    )

    output_path = "outputs/final_output.jpg"

    cv2.imwrite(

```

```
        output_path,  
        result  
    )  
  
    return {  
        "status": "success",  
        "output": output_path  
    }
```

Out put

